Twenty-First Implementation Iteration: Priority-Based Allocation Strategy

with Strategy Selection Through the Allocation Strategy Factory

Software Design / Iteration Report

Name: Najib Attar
Date: April 2026

1. Iteration Goal

The goal of this twenty-first implementation iteration is to make request priority affect actual allocation behavior by introducing a new concrete allocation strategy.

In earlier iterations, the prototype already supported request priority values through the request model, and the Strategy Pattern was already introduced for allocation algorithms. The system could select strategies through the `AllocationStrategyFactory`, but only the greedy strategy was available. As a result, priority existed as request data, but it did not yet influence the order in which requests were processed.

In this iteration, a new `PriorityAllocationStrategy` is added. This strategy processes loaded requests in descending priority order instead of raw CSV order. This demonstrates that the current Strategy + Factory design can be extended with new allocation behaviors while preserving the existing rule engine, request model, and CSV-based workflow.

2. Changes from Iteration 20

Compared with the twentieth iteration, the following changes were introduced:

- Added a new `PriorityAllocationStrategy` class
- Implemented priority-based batch request processing through the `IAllocationStrategy` interface
- Updated `AllocationStrategyFactory` so that it can return either greedy or priority-based strategies
- Preserved safe fallback to the greedy strategy for unknown strategy names
- Used stable sorting so that requests with the same priority preserve their original CSV order as much as possible

- Preserved the existing request approval, rejection, and allocation creation logic

This iteration does not introduce best-fit selection, global optimization, or any change to the current requested-space allocation model.

3. Scenario

The prototype starts by loading users, spaces, and requests from external CSV files through the `DataController`. After loading is completed, the main program creates an `AllocationService` and selects the active allocation strategy through the `AllocationStrategyFactory`.

When the priority strategy is selected, the strategy receives the full loaded request collection and creates an internal ordered view of the request pointers. It then sorts the requests by descending priority using the priority values already stored in each request. Requests with the same priority keep their original relative order through stable sorting.

After ordering is determined, the strategy processes each request one by one. It uses the existing rule engine facade to check whether the request satisfies the implemented rules. If the request passes, it is approved and allocations are created immediately. If it fails, it is rejected and the rejection reason is stored. At the end of execution, the system exports:

- accepted allocations to `allocations.csv`
- full processed request outcomes to `request_results.csv`
- request lifecycle history together with each exported request result

4. Modules and Their Tasks

A. Strategy Interface Module

- Continue defining the common interface for allocation algorithms through `IAllocationStrategy`
- Support both individual request processing and batch request processing
- Allow different allocation strategies to be used without changing client-side logic

B. Greedy Allocation Module

- Continue providing the original greedy behavior
- Process requests in the order they are loaded from CSV

- Approve or reject immediately based on the rule engine result

C. Priority Allocation Module

- Process loaded requests in descending priority order
- Preserve original order among requests with the same priority
- Use the same approval, rejection, and allocation creation logic already used by the greedy strategy

D. Allocation Strategy Factory Module

- Continue selecting strategies through the `IAllocationStrategy` interface
- Return either `GreedyAllocationStrategy` or `PriorityAllocationStrategy`
- Safely fall back to greedy when an unknown strategy name is requested

E. Allocation Service Module

- Continue coordinating allocation processing
- Continue owning the allocation list
- Continue using the factory-selected strategy without depending directly on a concrete algorithm class

5. Iteration Comparison

Iter.	Requests	Resources	Main Design Change
17	Same + history log	Class + Lab + Meet	Request lifecycle history logging + export
18	Same + strategy-based processing	Class + Lab + Meet	Allocation Strategy Pattern + Greedy strategy
19	Same + explicit greedy batch flow	Class + Lab + Meet	Greedy batch processing moved into strategy
20	Same + factory-based strategy selection	Class + Lab + Meet	Singleton strategy factory + interface-based selection
21	Same + priority-based processing	Class + Lab + Meet	Added `PriorityAllocationStrategy` through factory-selected strategy architecture

6. Updated Class Diagram

Figure 1 shows the updated class diagram including the new `PriorityAllocationStrategy` and the extended factory support for multiple concrete strategies.

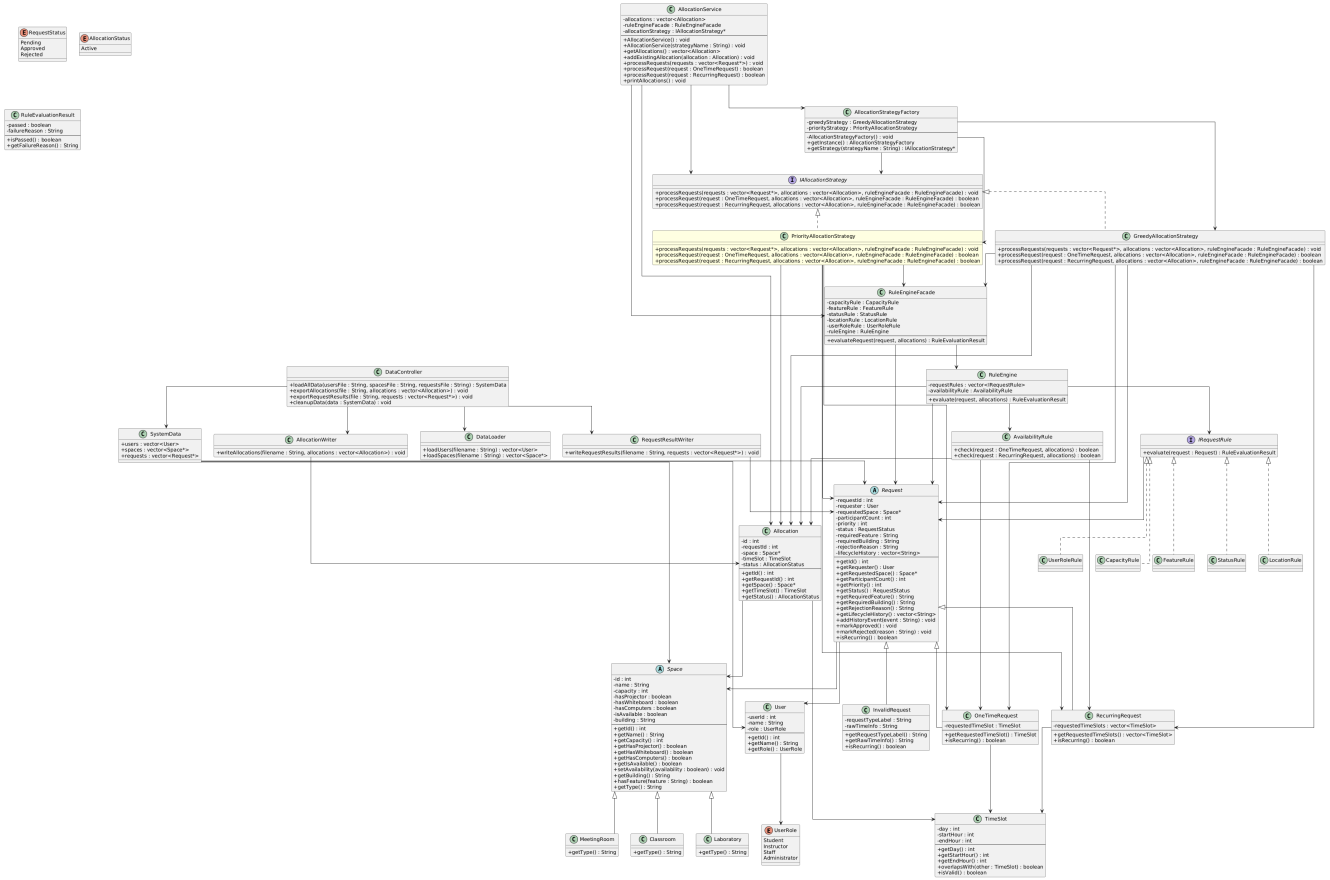

Figure 1: Updated Class Diagram with Priority-Based Allocation Strategy

7. Updated Workflow Diagram

Figure 2 shows the updated workflow. The system still loads requests through the same CSV-based process, but when the priority strategy is selected, request processing order is determined by priority rather than raw input order.

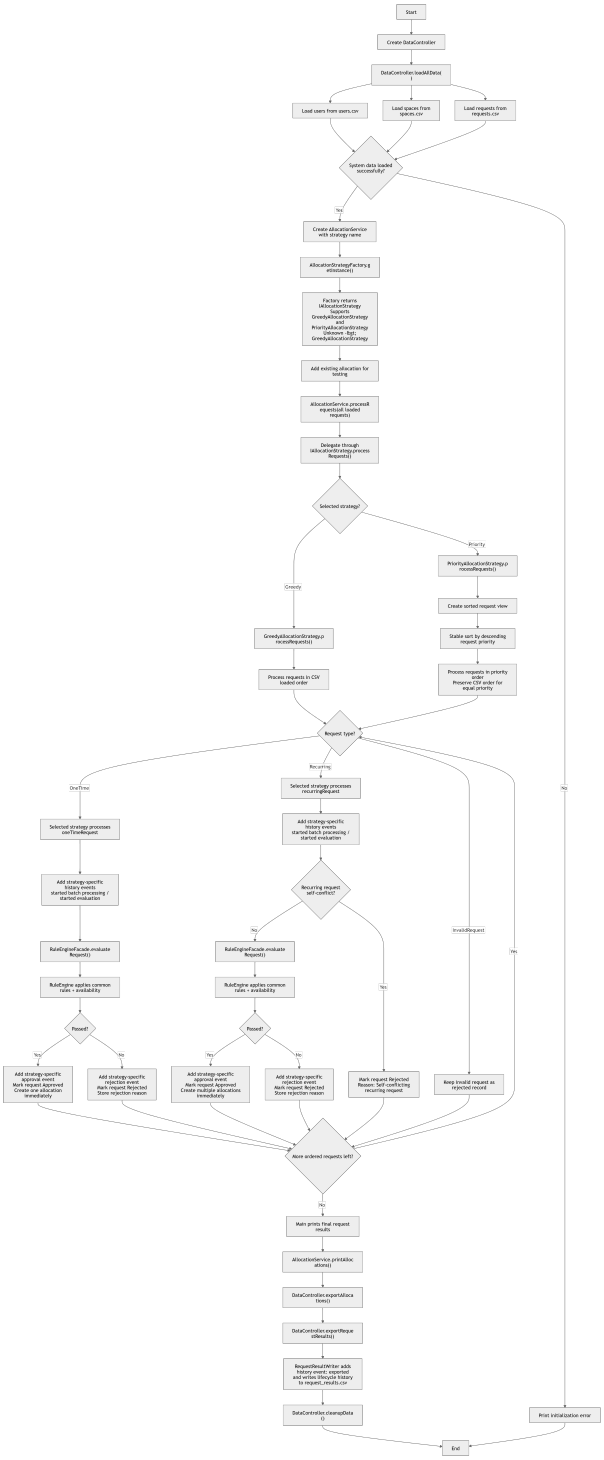

Figure 2: Workflow with Priority-Based Request Processing

8. Prototype Implementation

The C++ prototype was extended by adding `PriorityAllocationStrategy` as a new concrete implementation of `IAllocationStrategy`. This strategy receives the loaded request list, creates a sorted view of the request pointers, and processes them in descending priority order. Stable sorting is used so that equal-priority requests remain in their original CSV order as much as possible.

The current rule evaluation logic was not changed. The strategy still uses the existing `RuleEngineFacade`, and once a request is selected for processing, it follows the same approval and rejection behavior as before. If the request passes the rules, allocations are created immediately for the exact requested space. If the request fails, the request is rejected and the failure reason is preserved.

The `AllocationStrategyFactory` was updated so that it can now return either `GreedyAllocationStra` or `PriorityAllocationStrategy` through the base strategy interface. Unknown strategy names still fall back safely to the greedy strategy. This keeps the system extensible and shows that the current strategy-selection architecture can support additional allocation algorithms in later iterations.

This iteration does not implement best-fit allocation, global optimization, or any change in the request or CSV format. It only introduces a second concrete strategy that uses the already existing request priority value during batch processing.

9. Summary

This twenty-first implementation iteration makes request priority affect actual allocation behavior by introducing `PriorityAllocationStrategy`. The new strategy processes requests in descending priority order while preserving original order for equal-priority requests.

This iteration keeps the existing rule evaluation, request approval and rejection logic, exact requested-space allocation behavior, and CSV-based input/output unchanged. It also demonstrates that the Strategy + Factory architecture introduced in earlier iterations can now be extended with additional allocation strategies without major changes to client code.